

GeoJSON

GeoJSON is an open standard format for geographical information systems (GIS), supported by most GIS applications, GPS software and Web-based mapping applications. Many public data portals provide geographical data in GeoJSON or derived formats such as TopoJSON. GeoJSON files contain geographical coordinates for points and shapes. You can use GeoJSON to draw maps using vector graphics.

Most of the GeoJSON files we will use in this chapter contain a `FeatureCollection` object, with the general structure shown below. Its data is stored in the feature property: an array of Feature objects, each containing properties and geometry members:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "id": "MDG",
      "properties": { "name": "Madagascar" },
      "geometry": {
        "type": "Polygon",
        "coordinates": [
          [
            [44, -25], [43, -22], [45, -20], [44, -18], [45, -16], [47, -15],
            [49, -12], [50, -15], [47, -25], [45, -26], [44, -25]
          ]
        ]
      }
    }, { "type": "Feature", ... } // ... more feature objects
  ]
}
```

There are several GeoJSON files in the `data/` directory. The code above is from `data/africa/africa-madagascar.geojson`.

GeoJSON geometries

The most important object in a GeoJSON feature is the geometry object. It contains two properties: `type` and `coordinates`. The `coordinates` array is a collection of *positions*, which are `[longitude, latitude]` pairs (in this order). The specific structure of the `coordinates` array depends on the type of the shape.

Primitive geometries

There are three standard shapes in GeoJSON, which can be represented by *primitives* or *multipart* geometries. Primitive shapes are listed in *Table 1*.

Type	Description
Point	A <i>single position</i>, which is a 2-element array of the form: <i>[longitude, latitude]</i>, where <i>longitude</i> and <i>latitude</i> are floating point numbers in the ranges <i>[-180,180]</i> for <i>longitude</i>, and <i>[-90,90]</i> for <i>latitude</i>. Example: <pre>{ "type": "Point", "coordinates": [36.8484, 174.7633] }</pre>
LineString	An array of <i>two or more positions</i>. A line or open path with a single beginning and end. Example: <pre>{ "type": "LineString", "coordinates": [[-17,16],[-16,17],[-14,17],[-13,16],[-12,15],[-12,13]] }</pre>
Polygon	A collection of one or more <i>linear rings</i>: a sequence of <i>at least four positions</i>, where <i>the first and last positions are identical</i>. Linear rings are the boundaries of closed shapes (e.g. countries). Winding order is important: if a <i>Polygon</i> contains holes (e.g. lakes), their linear rings rotate in the opposite direction of the nearest outer boundary (e.g. country boundaries are counterclockwise, lakes are clockwise, islands in lakes are counterclockwise, and so on). Example: <pre>{ "type": "Polygon", "coordinates": [[[34,31],[34,27],...,[29,31],[34,31]], // outer [[35,-12],[34,-9],...,[35,-12],[35,-12]], // hole [[29,-4],[29,-6],[31,-9],[30,-6],[29,-4]] // hole] }</pre>

Table 1 – GeoJSON standard types: primitive geometries

The primitive GeoJSON shape types are illustrated in *Figure 1*:

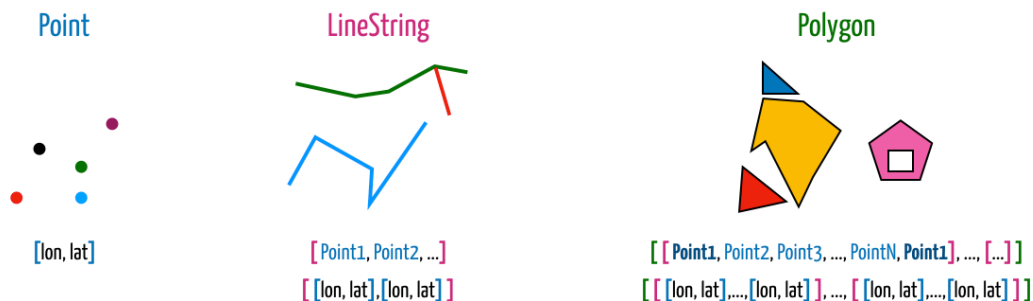


Figure 1– Examples of GeoJSON primitive types. The illustrations show five distinct *Points*, three distinct *LineStrings*, and four distinct *Polygons*. One of the polygons contains a hole.

Note that although a *LineString* requires only two *positions*, it can (and usually does) contain many more. Positions of different shapes may match, but that doesn't connect them in any way. In the illustration above the same position is part of the green and red *LineStrings*, but each is part of a different feature object and can have their own properties.

Multipart geometries

Multipart geometries group primitive geometries in a single feature. *MultiPoint* can be used to represent positions that belong together, sharing the same properties and never treated as individual points (e.g. a star cluster in a map of the sky). A *MultiLineString* represents multiple lines that always are treated as a single entity (e.g. a river basin with unnamed tributaries). A *MultiPolygon* can group a collection of shapes (e.g. a country and its islands) and treat it as a single feature object with the same properties. GeoJSON's types for multipart geometries are listed in *Table 2*.

Type	Description
MultiPoint	An array of <i>Point</i> objects. The coordinates member of a <i>MultiPoint</i> is an array of <i>positions</i> . Example: <pre>{ "type": "MultiPoint", "coordinates": [[32,-17],[32,-18]] }</pre>
MultiLineString	An array of <i>LineString</i> objects. Example: <pre>{ "type": "MultiLineString", "coordinates": [[[31,31],[31,30],[32,2],[33,1],[33,0]], [[33,16],[34,14]]] }</pre>
MultiPolygon	An array of <i>Polygon</i> objects. In the <i>MultiPolygon</i> below there are two polygons. The first one consists of two <i>linear rings</i> : <pre>{ "type": "MultiPolygon", "coordinates": [[[[34,31],[34,32],[33,32],[34,31]], // polygon 1 [[35,-12],[36,-12],[35,-14],[35,-12]]], // polygon 1 [[[44,-25],[43,-22],[45,-20],[44,-25]]] // polygon 2] }</pre>

Table 2 – GeoJSON standard types: multipart geometries

The multipart geometries supported in GeoJSON are illustrated in *Figure 2*.

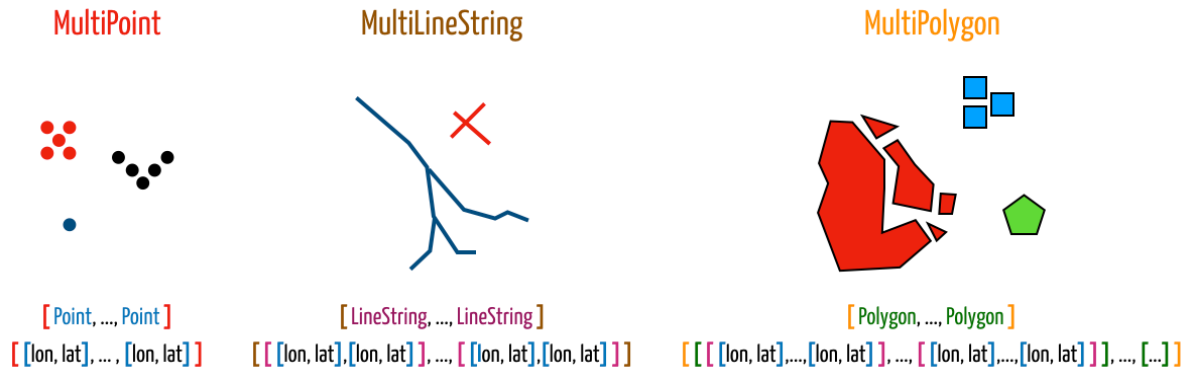


Figure 2 – Examples of GeoJSON multipart types. The illustrations show three distinct *MultiPoint* objects, two *MultiLineStrings* and three *MultiPolygons*.

Drawing geometries as paths

We have already used GeoJSON to draw a world map containing over 200 geometries, but they were all either *Polygon* or *MultiPolygon*. Let's try rendering the other types. The simplified GeoJSON file shown below (`data/africa/africa-simple.geojson`) contains four different geometry types: *MultiPolygon*, *LineString*, *MultiLineString* and *Point*, each one inside a separate *Feature*.

```
{ "type": "FeatureCollection",
  "features": [
    { "type": "Feature",
      "properties": {},
      "geometry": {
        "type": "MultiPolygon",
        "coordinates": [
          [[[34,31], ..., [34,31]], [[35,-12], ..., [35,-12]],
            [[32,-2], ..., [32,-2]], [[29,-4], ..., [29,-4]]],
          [[[44,-25], ..., [44,-25]]]
        ]
      }
    }, { "type": "Feature",
      "properties": { "id": "river1", "name": "Nile" },
      "geometry": {
        "type": "MultiLineString",
        "coordinates": [[[31,31], ..., [33,0]], [[33,16], ..., [37,12]]]
      }
    }, { "type": "Feature",
      "properties": { "id": "river2", "name": "Congo" },
      "geometry": {
        "type": "LineString",
        "coordinates": [[12,-6], ..., [29,-9]]
      }
    },
    // ... many more features with LineString or MultiLineString geometries
  ]
}
```

```

    { "type": "Feature",
      "properties": { "id": "city1", "name": "Lagos" },
      "geometry": {
        "type": "Point",
        "coordinates": [3,7]
      }
    },
    // ... many more features with Point geometries
  }
]
}

```

You really don't have to do anything different to draw *LineStrings* and *Points*. The `d3.geoPath()` function also takes care of that for you, but you might wish to choose an outline color for the lines, and make sure their fill is "none".

For this example, CSS will only be used to set the color for the strokes:

```
.continent { stroke: white }
```

Since the `d3.geoPath()` generator will render lines as paths, they need to be identified so that their fill style is configured to "none". You can create a simple JavaScript function to identify lines:

```

const isLine =
  d => d.geometry.type == 'LineString' || d.geometry.type == 'MultiLineString';

```

The `geoPath` function will automatically render points as circular paths, with radius configured by the `pointRadius()` method. The following code draws all the features and uses scheme-selected colors for points and shapes (see the complete code in `GeoJSON/1-lines-africa.html`):

```

const color = d3.scaleOrdinal(d3.schemeCategory10);
const svg = d3.select("body").append("svg").attr("width", 960).attr("height", 500);
const geoPath = d3.geoPath().projection(d3.geoMercator()).pointRadius(3);

d3.json('../data/africa/africa-simple.geojson').then(function(data) {
  svg.selectAll("path.continent")
    .data(data.features)
    .join("path").attr("class", "continent")
    .style("fill", (d,i) => isLine(d) ? "none" : color(i))
    .attr("d", geoPath);
});

```

The result is shown in *Figure 3*.



Figure 3 – A simple GeoJSON file containing Polygons, LineStrings and Points rendered by D3.
Code: [GeoJSON/1-lines-africa.html](#)

Drawing points and text

The function created with `d3.geoPath()` only generates path strings, so they are only good for `<path>` elements. If you want to draw points using other shapes, such as `<circle>` and `<rect>`, you need to translate every pixel position from geographical to pixel coordinates. This is easily done with the path's *projection* function. Projections work like two-dimensional scale functions, initialized with a default pixel range of `[900,500]`:

```
const projection = d3.geoMercator();  
const pixels = projection([100,-50]); // returns [746.94, 484.58]
```

For this example, the data needs to be split into separate datasets so that continent outlines, rivers and cities can be treated differently. They are filtered by `geometry.type`:

```
const rivers = data.features.filter(d => d.geometry.type=='LineString'  
                                     || d.geometry.type=='MultiLineString');  
const continent = data.features.filter(d => d.geometry.type=='Polygon'  
                                           || d.geometry.type=='MultiPolygon');  
const cities = data.features.filter(d => d.geometry.type=='Point');
```

Now configure some CSS styles for the text and each class:

```
text {  
  font-family: "Arial Narrow", sans-serif;  
  font-size: 10px;  
  text-anchor: middle;  
  alignment-baseline: hanging;  
}  
.continent { fill: #94923c; }  
.river { fill: none; stroke: aquamarine; }  
.city circle { fill: yellow; stroke: black; }
```

Since the river and continent outlines are rendered as paths in the same projection, the `geoPath` function call is reused with identical syntax in each selection (only the data is different):

```
svg.selectAll("path.continent").data(continent)
  .join("path").attr("class","continent").attr("d", geoPath)
svg.selectAll("path.river").data(rivers)
  .join("path").attr("class","river").attr("d", geoPath)
```

The location of a point representing a city is obtained with the `projection()` function configured for the `geoPath()`. It converts *Point* coordinates in `[longitude,latitude]` format to Cartesian `[x,y]` coordinates according to the projection. These coordinates are used by an SVG `translate` transform to place the points:

```
svg.selectAll("g.city")
  .data(cities)
  .join("g").attr("class","city")
  .attr("transform",
    d => `translate(${ [geoPath.projection()(d.geometry.coordinates)] })`)
  .each(function(d) {
    d3.select(this).append("circle").attr('r', 3);
    d3.select(this).append("text").attr('y', 2).text(d.properties.name);
  });
```

The result is shown in *Figure 4*. The full code is in `GeoJSON/2-points-africa.html`:



Figure 4 – Using a projection function to position cities and text labels.
Code: `GeoJSON/2-points-africa.html`.

Try these examples with other GeoJSON files. If your code doesn't use anything from the `properties` object, it should work with few or no changes.